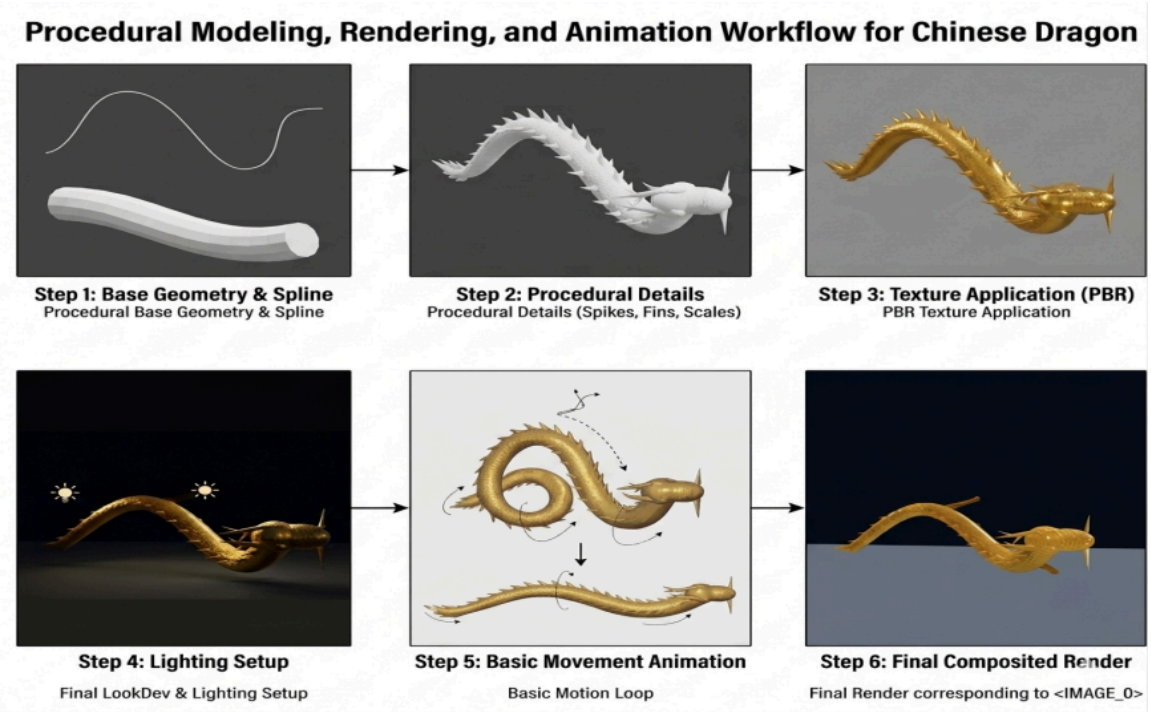
Chinese Dragon Project Writing Report

Contributors: WeiHao Dong, Donghui Li, Junjie Ma

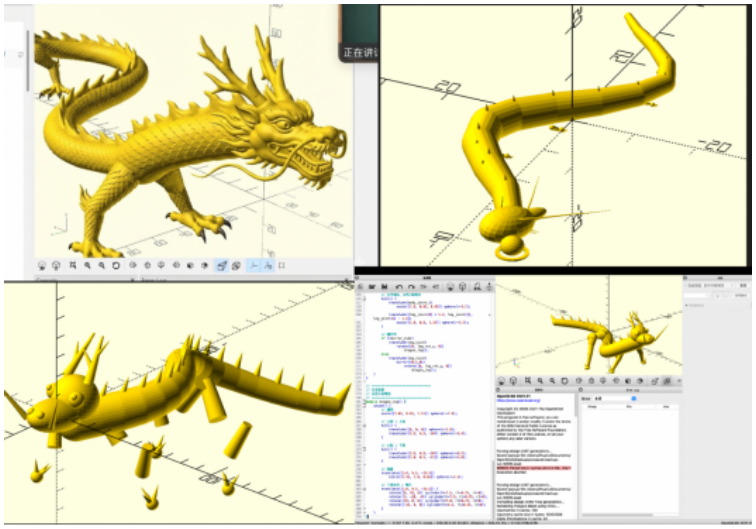
I. Introduction

Our project is a collaborative work based on the creation of a stylized Chinese dragon. As a group of three, we divided the project into three main parts: modeling, animation, and rendering. Each member was responsible for one major area, but throughout the whole process, we kept communicating closely, discussing problems together, and giving feedback to each other. We also worked together on proofreading and improving the final presentation materials, so that the final result would be more complete and consistent. In this presentation, we will introduce our design idea, workflow, individual contributions, and outcome.



II. Models

3D Modeling Optimization



Component	Main Technique	Implementation Notes
Head	sphere / cylinder / translate / rotate	The snout, back of the head, and brow ridge are formed by combining spheres, then horns, whiskers, and nostril details are added to make the model more recognizable as a Chinese dragon.
Body	Curve functions + hull()	Cross-sections are placed along a non-linear curve, and hull is used to smoothly connect adjacent segments to form a winding continuous body.
Limbs	Modular leg component	The front and back legs are generated from a shared module, with parameters used to distinguish their length, angle, and foot position.
Attachment logic	Body-curve binding	The legs are no longer attached using fixed straight-line coordinates. Instead, their attachment points are computed automatically from the body's y / z / radius at the corresponding x position.
Export	Switch variables + STL	Boolean switches such as SHOW_HEAD and SHOW_BODY are used to control visible parts, and the currently visible geometry is then exported.

When previewing the model, the preview speed is accelerated by reducing the value of $fn. After confirming that the model is error-free, the value is reset to 60 for the final export to produce a smoother result. Subsequently, toggle parameters are used to control the display and export of individual modeling components, such as the head, body, front legs, and hind legs. The entire model is set to gold to facilitate the observation of the model's outline and structural relationships within OpenSCAD. The bending direction of the body is controlled using sine

functions: body_y(x) governs lateral sway, and body_z(x) controls vertical undulation, creating a continuous, winding trajectory for the dragon's body. The body radius tapers gradually along its length, resulting in a thicker front section and a thinner tail, which conforms to the natural form of a dragon's body. In terms of specific implementation, the body is not constructed from a single cylinder; instead, multiple spheres are arranged along a curved path, and adjacent segments are smoothly connected using the hull() function. Dorsal spines are added along the top of the body to enhance its outline characteristics and decorative effect.

III. Animation

When creating the animation, it focused on the rotation animation. Meanwhile, based on the animation, further developed the zoom-in and zoom-out extension to observe the Chinese dragon's appearance and textures. To implement the rotation, use sine and cosine equations to vary the camera position along the x- and z-axes over time.

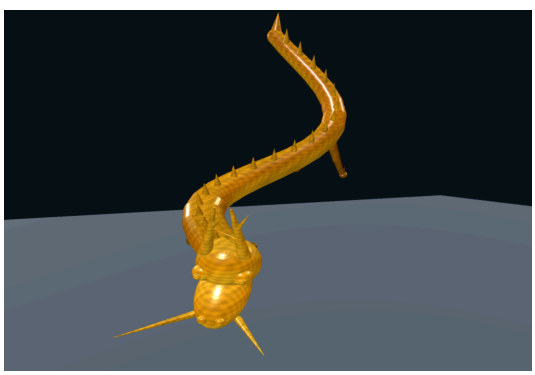
```
camera.position.x = Math.cos(timer) * radius;
camera.position.z = Math.sin(timer) * radius;
```

When creating the zoom-in and zoom-out animation, let the equations that determine the x- and z-axes multiply a radius that varies with the cosine range. The cosine range is between -1 and 1, so it can provide a camera distance to move closer and farther from the model over time. Also, change the y-axis camera position so it will mainly focus on the area we want the audience to observe (in this project, it focuses on the Chinese dragon's head and part of its body). The if-else loop is very helpful to determine when to switch the camera position and zoom in and zoom out.

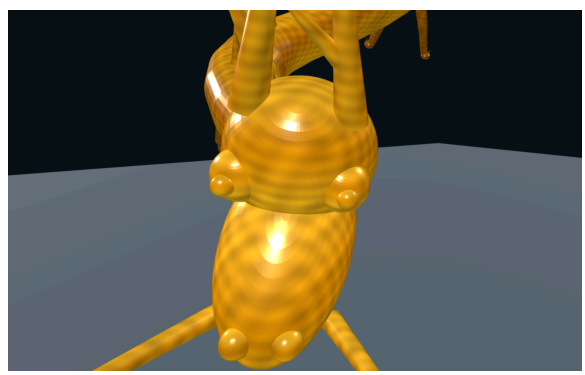
```
if (cycle < 6.8) {
  radius = 180 + Math.cos(timer) * 40;
  camera.position.y = 60;
} else {
  radius = 120 + Math.cos(timer) * 40;
  camera.position.y = 30;
}
```

When the time reaches the preset cycle threshold in the if-else loop, it will switch the camera position for the y-axis and change the radius that is multiplied by the x- and z-axes. The cycle uses the modulus operator $a \% b$ to return the remainder of a divided by b. So that after a specific time period, it can switch the second camera position back to the first camera position. The problem encountered while developing the animation is that it is hard to coordinate a better camera position when rotating the model and adjusting the dragon to a suitable position and orientation. The technique is continuously adjusting and proofreading the parameters to make the dragon appear to be flying in the sky above.

First Camera Position



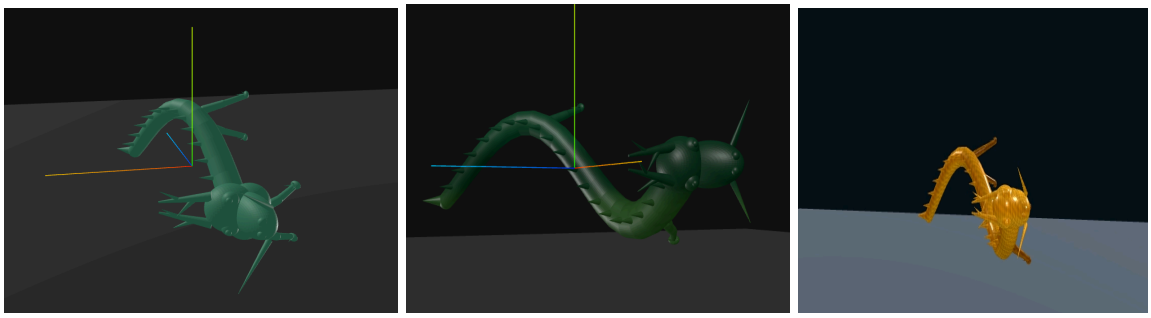
Second Camera Position



IV. Lighting and Texture Rendering

I was mostly in charge of the dragon's color scheme, lighting configuration, and surface texture effects in THREE.js throughout the rendering phase. I first decided on a green color scheme because I wanted the dragon to seem alive and natural. I didn't use a single, consistent color across the model to prevent it from seeming flat. To make certain parts look darker and

others brighter, I instead employed shader-based color variation. For instance, I created subtle changes across the dragon's body using expressions like dragonColor *= mix(0.92, 1.10, bodyGradient); and variables like baseColor, bellyColor, and accentColor. To make the shape and curving structure easier to see, I also incorporated directed and natural light. After comparing screenshots and evaluating the outcome, I saw that the green version had an odd appearance and did not closely match the classic Chinese dragon visual style. I modified the general color scheme to a golden-yellow design as a result. In the finished form, I utilized brighter highlights for the top surface, a softer pale-gold tone for the underbelly, and a stronger gold tone for the main body. As a result, the model seemed decorative and more in line with the idea of a stylized Chinese golden dragon. Instead of just using flat colors, I created a shader-based scaling effect to further enhance the surface look. To produce scale-like variation across the body, for instance, I used code like float scalePattern1 = 0.5 + 0.5 * sin(vPos.x * 1.8) * sin(vPos.z * 3.1). Therefore, the finished render resembled a dragon with more distinct texture, lighting, and surface hierarchy rather than a simple imported STL model.



V. Conclusion

The Chinese dragon project involves many aspects of knowledge to use the materials and technologies. It’s a new experience for people who have never used the tools, such as the OpenSCAD or THREE.js, to create models, animations, and texture renderings. This project

allows us to gain a deeper understanding of the professional knowledge and concepts within the field of 3D modeling and coding. In the future, these fundamental skills can be used to help develop more detailed models, richer animations, and more exquisite textures. Teamwork has honed our ability to divide tasks and coordinate with one another. The collaboration distribution is divided into three parts:

1. WeiHao Dong: Chinese dragon Modeling; oral presentation/writing report introduction and modeling sections; proofreading.
2. Donghui Li: Chinese dragon texture rendering and lights; oral presentation/writing report lighting and texture rendering sections; proofreading.
3. Junjie Ma: Chinese dragon animation; oral presentation/writing report animation and conclusion sections; proofreading.